

Common Pitfalls and Anti-Patterns

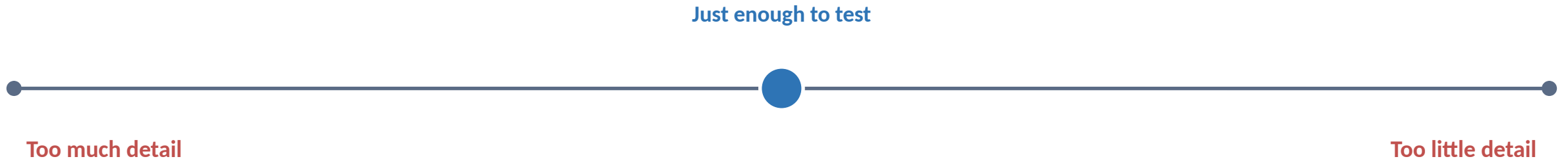
Most anti-patterns are earlier shortcuts catching up with you.

What You'll Learn

- 1 Recognize over-specifying and under-specifying and find the right level of detail
- 2 Weigh token and cost tradeoffs against rework avoided
- 3 Learn from documented, framework-specific pitfalls in both tools

THE BALANCE

Over- and Under-Specifying



Both directions push cost somewhere, just at different points in the lifecycle

REVISITED

Token and Cost Tradeoffs



Treat it as a tradeoff to manage, not a cost to minimize blindly

WHY IT MATTERS

Most anti-patterns are visible in hindsight, rarely in the moment.



Gate skipped
seems fine



Weeks pass
nobody notices



Cost surfaces
downstream

APPLYING THE CONCEPTS

GitHub Spec Kit

Where Teams Get Stuck

Stale checklist: Treating a passed checklist as permanently done rather than re-running it after changes: a checklist reflects the spec at the moment it was generated, and later edits can silently invalidate it, so a checklist needs revisiting whenever the spec it was built from changes.

Ignoring analyze: Ignoring `/speckit.analyze` findings instead of returning to the step that owns them (specify or clarify for requirement problems, plan for design problems) lets small inconsistencies compound into much larger ones by the time implementation is done.

APPLYING THE CONCEPTS

OpenSpec

Documented Failure Modes

From OpenSpec's docs: "Change not found," "No artifacts ready," and "Artifacts not generating properly" are named directly in OpenSpec's own troubleshooting guide. Thin project context in config.yaml is a common root cause, so the fix is often as simple as filling in more project context.

```
/opsx:ff
```

Fast-forward generates every artifact at once. Convenient, but it skips the incremental review that `/opsx:continue` gives you, an under-specification risk if nobody reads each artifact before the next is generated on top of it.

Module 8 at a Glance

	GitHub Spec Kit	OpenSpec
Common pitfall	Stale checklists or ignored analyze findings	Over-using /opsx:ff without review
Root cause	A review step was skipped	A review step was skipped
Documented by	Workflow reference (checklist, analyze)	OpenSpec's own troubleshooting guide

Key Takeaways

- Anti-patterns are rarely new mistakes; they're earlier shortcuts catching up with you later in the lifecycle.
- Every framework gate you skip is a bet that nothing needed catching, and that bet doesn't always pay off.
- Case studies at scale tend to share one pattern: the team that skipped a gate is the team that got surprised later, often at a worse time than if they'd caught it early.